Learning AI Agent Design Through a PEAS-Guided Immersive VR Experience

Fixed block state machine + block-based coding

Platform: Unity 6 / Meta Quest

Multiplayer: Photon Fusion 2 (Shared Authority)

Voice: Photon Voice 2

Study Design:

https://doc.photonengine.com/arvr/v1/cross-platform-samples/fusion-vr-training

N = 56 participants - 28 pairs

VR condition: 14 pairs

2D desktop condition: 14 pairs

Part A - Symmetric Collaborative Architecture

A1. Network Topology and Session Structure

The system uses Photon Fusion 2 with Shared Authority topology — the same topology used in the drone project. There is no dedicated host; both participants are peers. In the symmetric model, both participants hold equal StateAuthority potential over all shared objects. Authority transfers dynamically: whichever client initiates a grab or UI interaction claims StateAuthority on that object for the duration of the interaction.

> 🔧 **DRONE PROJECT REFERENCE**
>
> **Feasibility: Direct reuse — no modification needed**
>
> Module to borrow: ConnectionManager addon + XRShared (from /Photon/FusionAddons/)
>
> How to adapt: Copy the ConnectionManager prefab from the drone project. It handles room creation, both participants joining the same Photon room, and spawning each player's NetworkRigVariant. Replace the drone's LearnerActivitiesPrefab with your BlockCanvasPrefab. Both players spawn with identical rig configurations — no role flag is set.
>
> Code note: ConnectionManager.gameObject → set learnerActivitiesPrefab = BlockCanvasPrefab

Network Object	StateAuthority Owner	What It Controls	Drone Equivalent
SharedRuleSet	Whichever client last modified it (transfers on write)	The active JSON rule array driving robot behavior	ActivitiesAvailability NetworkDictionary
ObservationLog	Either client (writes are additive; no conflict)	All ✓/✗/? stamps with PlayerID tags	LearnerActivityTracker Progress float
PerformanceDashboard	Scene object (no owner)	Live scores on four dimensions	ScoreBoard scene object
PlayerRig_A / PlayerRig_B	Each player's own client	VR rig position, hand controllers, head	NetworkRigVariant per learner
RobotSimulation	Whichever client last triggered an action (transfers)	Robot NavMesh position, animation state	DroneControl NetworkTransform

A2. Symmetric Access

Both participants' grab colliders are always active on all blocks. Both participants can press Robot POV, stamp the observation log, use vibe coding, and trigger trace highlights.

Simultaneous Grab Conflict Resolution

When both participants attempt to grab the same block at the same moment, Fusion's StateAuthority transfer resolves the conflict automatically: the first client to call RequestStateAuthority() wins. The other participant's grab attempt fails silently — their hand passes through the block (0.3s) and the block remains with the first grabber.

This is the same conflict resolution mechanism the drone project uses when multiple learners reach for the same drone part simultaneously. No additional code is required.

🔧 **DRONE PROJECT REFERENCE**

Feasibility: Direct reuse — drone's grabbing system is identical

Module to borrow: XRShared local rig grabbing + NetworkTransform on each block prefab

How to adapt: The drone uses local rig grabbing where any learner can grab any part. Each drone part has a NetworkTransform that syncs position to all clients. Apply the same component to each block prefab. When either participant grabs a block, that client calls RequestStateAuthority on the block's NetworkObject. The other participant sees the block moving via NetworkTransform interpolation.

Code note: block.GetComponent<NetworkObject>().RequestStateAuthority(); // called on grab by either player

A3. Shared Block Canvas — Network Sync

When either participant places, moves, or removes a block, the change must be visible to the other within one network tick (~50ms on LAN, ~100ms on typical WiFi). The SharedRuleSet NetworkArray is the single source of truth. Any client can write to it when they hold StateAuthority.

🔧 **DRONE PROJECT REFERENCE**

Feasibility: Adapt from drone — medium effort

Module to borrow: LearningManager.ActivitiesAvailability (NetworkDictionary) → adapt as SharedRuleSet (NetworkArray)

How to adapt: The drone's ActivitiesAvailability is a NetworkDictionary<int, ActivityStatus> that all clients read but only StateAuthority modifies. Replace with NetworkArray<RuleData> of capacity 20. When either participant snaps a block into a slot, that client transfers StateAuthority over SharedRuleSet and writes the new rule entry. Fusion's OnChangedRender fires on all clients, triggering a robot preview update.

Code note: [Networked, OnChangedRender(nameof(OnRuleSetChanged))] [Capacity(20)] public NetworkArray<RuleData> SharedRuleSet { get; }

RuleData Struct (Fusion networked struct)

```
public struct RuleData : INetworkStruct {
  public NetworkString<_16> SenseBlockId;
  public NetworkString<_16> CondBlockId;
  public NetworkString<_16> ActionBlockId;
  public NetworkString<_16> CtrlBlockId;          // nullable
  public NetworkBool Active;
  public NetworkString<_8> PlacedByPlayerID;  // for post-session analysis
}


void OnRuleSetChanged() { RobotSimulation.ReloadRules(SharedRuleSet); }
```

A4. Voice Communication

Voice is the primary coordination mechanism in the symmetric model. Because neither participant is constrained to a specific role, verbal negotiation — "I'll do the sensors, you watch the preview" — is how the pair naturally divides attention. This is a direct research observation opportunity.

> 🔧 **DRONE PROJECT REFERENCE**
>
> **Feasibility: Direct reuse — zero modification**
>
> Module to borrow: Photon Voice 2 + Dynamic Audio Group addon
>
> How to adapt: Copy the Photon Voice 2 integration from the drone project verbatim. The Dynamic Audio Group addon adjusts volume based on player proximity. No code changes needed — create a Voice AppId in the Photon Dashboard and paste it into Real Time Settings.
>
> Code note: // No code changes. Configure AppId in Real Time Settings only.

A5. Disconnection and Reconnection

> 🔧 **DRONE PROJECT REFERENCE**
>
> **Feasibility: Direct reuse — critical for study reliability**
>
> Module to borrow: Reconnection addon + Recoverable class + UserId in PlayerPrefs
>
> How to adapt: If either participant's Quest loses connection mid-session, the drone's reconnection system recovers their objects and progress automatically. Apply Recoverable component to: each player's NetworkRigVariant, the SharedRuleSet manager, and the ObservationLog. On reconnect, the player reclaims StateAuthority over their recoverable objects and the session continues.
>
> Code note: // Add Recoverable component to all persistent network objects. UserId auto-saved to PlayerPrefs on first join.

Part B - Design Flow

Both participants have identical affordances throughout all five steps. Either participant may initiate any action at any time. The pair coordinates through voice.

	Name		LOs
Step 1	Define your agent	Selects agent type, tasks, sets sliders - Reads environment labels aloud, confirms purpose statement	LO1, 2, 3
Step 2	Know your world	Selects sensors on panel - Narrates what robot can/cannot see	LO4, 5
Step 3	Give your robot a body	Selects actuators, resolves conflicts - Reads alignment check, flags mismatches aloud	LO6
Step 4	Program your robot	Places blocks, uses block based coding - Watches right-wall preview, stamps observation log	LO2, 7
Step 5	Test, debug and reflect	Executes debug moves in debug room - Triggers Robot POV during test run, leads PEAS reflection	LO3, 7, 8

Step 1: Define Your Agent

PEAS: Performance Measure (P) — partial
Learning Objectives: LO1 (define agent purpose), LO2 (begin PEAS), LO3 (goal vs. measure)
Competencies: C1 Recognizing AI, C8 Decision-Making, C17 Programmability

Purpose: Both participants enter the VR apartment together for the first time. Step 1 establishes the foundational design commitment that constrains every later decision. In the symmetric model, both participants interact with all UI elements. They must negotiate verbally when making joint selections.

1.1 - C1 Baseline: Data Captured

Three household objects float in the apartment. Both participants see them and independently classify each object using their own controller.

Object	Classification	Feedback
Robotic vacuum	AI	Both participants see their own green confirmation
Standard vacuum cleaner	Not AI	Both participants see their own red prompt
Smart thermostat	AI	Both participants see their own green confirmation

1.2 - Tasks, Performance Measurements and Environment

The agent type card and task panel appear. Both participants can interact with the selection UI. Both participants physically walk through the apartment simultaneously. Environment labels appear at key locations. Either participant may read labels aloud and point to locations using a controller. Both participants jointly confirm the purpose statement before proceeding.

#	Task Name	Robot Animated Preview
1	Vacuum the floor	Robot moves across floor grid; debris particles disappear
2	Pick up and sort scattered objects	Robot arm extends to collect items; deposits them into designated drop zones by object type (trash vs. belongings)
3	Detect and clean spills	Wet sensor flags spill cell; robot deploys mop attachment and stays on the cell until fully clean

4	Detect and avoid the moving cat	Cat crosses robot's path; robot stops, waits, then reroutes — cat continues moving uninterrupted
5	Slow down near participants	When a participant avatar is within close range, robot drops to slow speed — speeds back up once they move away
6	Return to charging dock when battery is low	Battery icon pulses red; robot abandons current task and navigates back to the dock in the corner

1.3 - Performance Measure Sliders

The slider panel appears. Both participants can adjust any of the four sliders. The robot preview responds in real time to every adjustment, visible to both participants simultaneously. When two opposing sliders conflict, the ERROR prompt appears for both players. Either participant may resolve it. Both participants must jointly confirm the configuration summary before proceeding.

Dominant Weight	Robot Color	Observable Behavior
Coverage high	Yellow	Robot methodically covers the entire floor grid before stopping
Safety high	Green	Robot slows near cat and NPC; gives furniture wide berth
Time high	Blue	Robot moves faster, skips secondary areas; countdown overhead
Hazard response high	Purple	Robot stops at every spill and cable before continuing

Step 2 - Know Your World

PEAS: Environment (E) + Sensors (S)
Learning Objectives: LO4 (characterize environment), LO5 (select sensors for task and environment)
Competencies: C14 Action & Reaction, C15 Sensors
Purpose: Both participants walk through the apartment, read environment labels, select sensors, and experience Robot POV mode. In the symmetric model, either participant can press the Robot POV button. The key learning moment — inhabiting the robot's sensor-filtered perspective — is available to both. Pairs may take turns in POV, or one may narrate while the other explores; the coordination is theirs to decide.

2.1 - Environment Characterization Walk-Through (~3 min)

Both participants walk through the apartment simultaneously. Floating environment labels appear at specific locations.

Location	Floating Tag	Floating Label Text
Kitchen (hidden spill)	Partially observable	'Your robot cannot see this until it arrives here.'
Roaming cat(https://assetstore.unity.com/packages/3d/characters/animals/mammals/free-chibi-cat-165490)	Dynamic environment	'This obstacle moves. Your robot's map from a moment ago may already be wrong.'
Cluttered living room	Continuous / non-deterministic	'There is no single correct path through here.'
Long hallway cable	Sensor-specific detectability	'A distance sensor may miss this. A floor map sensor would detect it.'
Charging dock	Goal localization	'Your robot needs to find this reliably from any location.'

2.2 - Sensor Selection

The sensor selection step uses a three-panel layout: The interaction produces three important learning moments. First, because students must open each task row and think about what that specific task requires before dropping a sensor, the task-sensor connection is made explicitly rather than by selecting from a general list. Second, when the robot preview updates in real time, students see the robot becoming capable in ways that are tied to their design choices. Third, there are four incorrect sensors without labeling

them as wrong, requiring students to read descriptions carefully and make a judgment about relevance to their task set.

2.2.1 - The left panel

- Shows the six required tasks as expandable rows. Each row has a drop zone where students drag sensor cards to assign sensors to that task.
- *Floating text: "Drag sensors from the library into each task that needs them. Your robot can hold 11 sensors maximum."*

#	Task Name	(Drop sensors here)
1	Vacuum the floor	SENSE_04; SENSE_05; SENSE_07
2	Pick up and sort scattered objects	SENSE_03; SENSE_04; SENSE_08; SENSE_15
3	Detect and clean spills	SENSE_04; SENSE_06; SENSE_07
4	Detect and avoid the moving cat	SENSE_01; SENSE_04; SENSE_12
5	Slow down near participants	SENSE_04; SENSE_09;
6	Return to charging dock when battery is low	SENSE_02; SENSE_03; SENSE_04

2.2.2 - The sensor library - The middle panel

- **The sensor library**, containing all 15 sensors (11 correct and 4 incorrect). Each displayed as a draggable block showing the sensor ID, name, and a plain-language description of what it detects and what it cannot detect.

Blocks	Sensor	Measures	Misses	Body Module	Task
SENSE_01	Animal detection sensor	Presence and position of tagged NPC_Animal objects within a 1.5 m radius	Humans, furniture, walls, only fires on the cat tag. Does not detect direction of travel.		4
SENSE_02	Battery sensor	Current battery level as a percentage: HIGH (>60%) / MEDIUM (30–60%) / LOW (<30%)	time remaining at current drain rate	battery bar on robot chest, pulses red at LOW	6
SENSE_03	Destination sensor	Distance and direction to a tagged goal location: SINK, BIN, or DOCK	Whether the destination is reachable; obstacles	directional arrow display on head screen	2, 6

			between robot and destination		
SENSE_04	Distance sensor	Distance to nearest obstacle in 4 directions (front, back, left, right) up to 5 m	Define the obstacle - cat, furniture, wall, and participant	sonar rings around robot base	1, 2, 3, 4, 5, 6
SENSE_05	Floor dirt sensor	Whether current grid cell is tagged DIRTY	object type; cells not directly underfoot	LED ring on undercarriage glows red on dirty cell	1
SENSE_06	Floor wet sensor	Whether current grid cell is tagged WET	Exact distances	LED ring on undercarriage glows blue on wet cell	3
SENSE_07	Floor map sensor	State of every grid cell in the apartment: CLEAN / DIRTY / WET / VISITED / UNKNOWN	Future state changes; moving objects		1, 3
SENSE_08	Grip sensor	Whether the mechanical arm is currently holding an object successfully	What the object is; whether it will drop if the robot moves too fast; grip pressure	LED on arm joint glows green when gripping	2
SENSE_09	Human proximity sensor	Presence and distance of tagged PlayerAvatar objects within a 2 m radius	The cat, furniture, walls, only fires on participant avatar tags.	blue proximity ring on robot chest	5
SENSE_10	Light level sensor	Detects ambient brightness in the current room in lux. Returns a reading from dark to bright.			wrong
SENSE_11	Object weight sensor	Detects the approximate weight of any object the arm is currently in contact with. Returns a weight value in grams.			wrong
SENSE_12	Session timer sensor	How long the current task has been running, in seconds	robot stuck in a loop (e.g. waiting for cat to move) has no way to give up and try	Timer display	4

			something else after a timeout		
SENSE_13	Sound level sensor	Detects ambient noise level in the apartment in decibels. Returns a loudness reading.			Wrong
SENSE_14	Stair edge detector	Detects whether the robot is approaching a stair edge or floor drop-off. Returns true when a drop is detected ahead.			wrong
SENSE_15	Vision sensor (object classifier)	Type and position of tagged objects in a 120° forward-facing cone: CUP, FOOD, TRASH, BIN, SINK	Exact distances; objects outside the cone; objects behind obstacles; untagged items	Blinking eyes on robot head	2

2.2.3 - The right panel

- Shows a live preview of the robot, with sensors appearing as mounted modules on the robot body each time a new sensor is added to any task.
- The slot counter at the top of the library panel displays how many unique sensors have been placed across all tasks — a student can assign the same sensor to multiple tasks without using additional slots, since the same physical sensor module serves all tasks simultaneously. The maximum is 11 slots, matching the number of physical mounting points visible on the robot preview, and the minimum is 8 slots.
- After at least 8 sensors have been placed, the confirm button activates and students proceed to the percept summary screen, which shows two columns: what the robot can now perceive, and what it cannot perceive given the sensors that were not selected. This summary is the bridge between sensor selection and the Robot POV mode that follows, making the bounded percept sequence concept explicit before students inhabit the robot's perspective.

Step 3 - Give Your Robot a Body - Actuators

PEAS: Actuators (A)

Learning Objectives: LO6 (select actuators and explain their function)

Competencies: C14 Action & Reaction

Purpose: The robot body has exactly 10 actuator mounting slots. The library contains 14 actuator cards. Students must fill all 10 slots before they can proceed.

3.1 - Room Layout: Actuator Selection Panel

Both participants are present simultaneously with identical affordances — either can interact with any panel at any time.

LEFT PANEL	CENTER PANEL	RIGHT PANEL
Task Reference Panel	**Actuator Library + Slot Counter**	**Robot Assembly View**
Six expandable task cards, carried over from Step 1. Each card shows: - Task name - Plain-language description of what the task requires the robot to physically do - A drop zone where actuator cards can be placed for reference The task cards are an organizational scaffold — not a validation mechanism. Any actuator can be dragged into any task row. The system does not restrict or confirm placements. **Floating prompt:** *"Think about what your robot needs to physically do to complete it. Drag the actuators it needs into the task row."*	14 actuator cards organized in functional groups. Each card shows: - Actuator name - What it physically does - What it cannot do - A small icon of where it mounts on the robot body No card is labeled correct or incorrect. Students must read and judge relevance themselves. **Slot counter:** A persistent counter at the top of the center panel: "X / 10 slots filled." Updates in real time as cards are placed or removed. The confirm button activates only when the counter reaches 10 / 10.	Assembly view of the robot body. Shows the robot as a 3D object that both participants can walk around and inspect. Updates immediately when any actuator card is placed or removed. **When a card is placed into a slot:** The corresponding physical module appears on the robot body and plays a brief single animation showing what it physically does — then settles into its mounted position. **When a card is removed:** The module disappears from the robot body immediately. No task behavior, no fail traps, no behavioral loops. The assembly view shows only what the robot currently has on its body — nothing more.

3.2 - Full Actuator Library

The library contains 14 actuator cards across five functional groups. 10 are required to complete all six tasks. 4 are plausible but not required for this specific task set — students must reason through the descriptions to determine which four to leave out.

Actuator	What It Does Physically	Cannot Do
Locomotion		
Wheel Drive	Spins the drive wheels to move the robot forward, backward, or in place. Speed is set in Step 4 programming logic — slow near people, fast in open areas.	*Cannot move laterally; turning radius is fixed.*
Body Rotation	Rotates the chassis left or right in place by a set angle (45°, 90°, or 180°) without forward movement.	*Cannot strafe or move diagonally; rotation is discrete steps, not continuous.*
Emergency Stop	Immediately halts all wheel motion in the current tick. Overrides any active movement command. Required when the robot must stop faster than a normal deceleration allows.	*Does not reverse; does not move sideways; robot stays exactly where it stopped.*
Cleaning Tools		
Vacuum Suction	Activates suction on the undercarriage. While active over a DIRTY cell, the cell changes to CLEAN after 1 second of contact.	*Cannot clean WET cells — liquid damages the suction mechanism. Cannot pick up large objects.*
Mop Pad Deploy / Retract	Lowers a mop attachment from the undercarriage. While deployed over a WET cell, the cell transitions to CLEAN after 2 seconds of contact.	*Cannot be active simultaneously with vacuum suction. Cannot clean DIRTY (dry) cells.*
Mechanical Arm		
Arm Extend	Telescopes the arm outward up to 0.6 m toward a tagged object. Arm aligns to object position automatically.	*Cannot reach objects further than 0.6 m; cannot extend through walls or furniture.*
Gripper Close (Grab)	Closes the gripper fingers around the nearest tagged object within reach. On success, the grip sensor returns true.	*Cannot grab objects the arm has not extended toward; cannot hold two objects at once.*

Gripper Open (Release)	Opens the gripper fingers, releasing the held object. Object drops to the surface below the arm's current position.	*Cannot place an object precisely — drops at arm's current position and falls to gravity.*
Arm Retract	Pulls the arm back to the folded home position flush against the robot chassis after use.	*Cannot retract while the gripper is still closed around an object.*
Docking		
Dock Connector Engage	When within 0.2 m of the charging dock, deploys a physical connector pin into the dock socket. Charging begins and battery level starts rising.	*Cannot engage from further than 0.2 m; robot must be precisely aligned with the dock socket.*
Additional Actuators		
LED Body Flash	Pulses the LED strip around the robot body in a specified colour: red (error), amber (caution), blue (carrying/mopping). Makes robot state visible to observers from any angle.	*Cannot display text; colour meaning is fixed by convention, not configurable.*
Bin Lid Open	Sends a signal to a smart bin in the environment. The bin lid opens automatically, ready to receive an object deposited by the arm.	*Cannot open the sink drain or any other container. Requires a smart bin in the environment — a standard open drop zone does not need this actuator.*
Speaker / Voice Output	Emits a pre-defined audio signal when triggered by a block — e.g., a beep when a task completes or a chime when battery is low.	*Cannot produce arbitrary speech; limited to a fixed library of tones and chimes.*
Arm Speed Control	A dedicated hardware module that adjusts how fast the mechanical arm extends and retracts — slower for fragile objects, faster for routine pick-up.	*Arm movement speed is already a programmable parameter set in Step 4 block logic. This module duplicates what the programming layer handles — it does not add a new physical capability.*

3.3 - Left Panel: Task Reference

The six task cards are an organizational scaffold. Participants open each card to read what the task physically requires, then use the center library to find matching actuators. The task rows have drop zones where cards can be placed for reference, but placement is unrestricted — the system does not validate which actuator belongs in which row.

Task	What the Task Requires the Robot to Physically Do	Actuators Required — and Why
1 — Vacuum the Floor	Move across the floor grid, change direction to cover all areas, and physically remove dirt from cells on contact.	Wheel Drive (move) Body Rotation (turn to cover grid) Vacuum Suction (clean dirty cells) Without Body Rotation the robot can only sweep one straight line. Without Vacuum Suction the robot navigates correctly but the floor stays dirty.
2 — Pick Up and Sort Scattered Objects	Navigate to each object, orient toward it, reach out and grab it, carry it to the correct destination, release it, and fold the arm back before moving to the next object.	Wheel Drive (navigate) Body Rotation (orient to object and destination) Arm Extend (reach) Gripper Close (grab) Gripper Open (release at destination) Arm Retract (fold back — without this, arm collides with doorframes during transit). Speed during carrying is set in Step 4 programming logic, not by a separate actuator.
3 — Detect and Clean Spills	Navigate to wet cell locations and deploy a cleaning mechanism that can handle liquid. Note: vacuum suction cannot clean wet cells.	Wheel Drive (navigate) Body Rotation (approach from any angle) Mop Pad Deploy/Retract (the only physical mechanism that cleans WET cells — vacuum suction is explicitly wrong here).
4 — Detect and Avoid the Moving Cat	Stop immediately when the cat crosses the path — faster than a normal deceleration. Then reroute around the obstacle.	Emergency Stop (halt in one tick — Wheel Drive cannot stop fast enough when a dynamic obstacle appears suddenly) Body Rotation (reroute after stopping) Wheel Drive (resume movement).

5 — Slow Down Near Participants	Reduce movement speed when a participant avatar enters the proximity zone, then resume normal speed when they leave. Speed modulation is handled in Step 4 programming — Wheel Drive's speed is a programmable parameter.	Wheel Drive (speed is set in programming logic — no separate slow actuator needed) Body Rotation (must still navigate while moving at reduced speed) No additional actuator beyond what Tasks 1–4 already require.
6 — Return to Charging Dock When Battery is Low	Interrupt the current task, navigate to the charging dock from any location in the apartment, and physically initiate charging on arrival.	Emergency Stop (interrupt current task cleanly) Wheel Drive (navigate to dock) Body Rotation (navigate around furniture en route) Dock Connector Engage (physically initiate charging — arriving at the dock without this does nothing; battery stays flat).

3.4 - Right Panel: Robot Assembly View

The assembly view shows the robot body as a 3D object that both participants can walk around and inspect in VR. It has exactly 10 mounting slots, shown as outlined attachment points on the robot surface. As actuator cards are placed, modules appear and fill those slots. As cards are removed, modules disappear.

The assembly view does not show task behavior, fail traps, or performance outcomes. It shows only what the robot currently has on its body. Task behavior is observed for the first time in Step 5 test run.

Actuator Selected	Module Added to Robot Body	Brief Animation on Placement	Unity Asset Link
Locomotion			
Wheel Drive 	Drive wheels appear on robot base. Wheel rotation animation activates while the card is being placed.	Wheels spin forward for 1 second then idle. Both participants can see the wheels are the primary movement mechanism.	https://assetstore.unity.com/packages/3d/props/low-poly-3d-wheel-model-pack-247391
Body Rotation	Sonar ring array appears around the robot base at ground level.	Robot chassis rotates 90° left, then 90° right, then returns to center. Sonar rings sweep during the rotation.	

Emergency Stop	Red halt indicator light appears on robot chassis front face.	Indicator flashes red twice, then dims to standby. No mechanical movement — this is a control signal module, not a mechanical attachment.	
Cleaning Tools			
Vacuum Suction 	Spinning brush module appears on undercarriage underside.	Brush spins at full speed for 1 second, then idles. Suction air particles briefly visible around the brush.	https://assetstore.unity.com/packages/3d/props/electronics/robotic-vacuum-cleaner-set-342428
Mop Pad Deploy / Retract 	Flat mop pad module appears below undercarriage in retracted position alongside the vacuum brush (separate slots — they do not overlap).	Mop pad lowers to floor contact position for 1 second, then retracts. Students can see that vacuum brush and mop pad occupy distinct physical locations on the undercarriage.	
Mechanical Arm			
Arm Extend 	Telescoping arm appears folded against robot's right side.	Arm telescopes outward to full extension (0.6 m), holds for 0.5 seconds, then retracts. Targeting reticle appears briefly at the extension endpoint.	https://assetstore.unity.com/packages/3d/characters/robots/delivery-boxy-gripper-robot-evolution-pack-cute-series-203640
Gripper Close (Grab)	Gripper finger assembly appears at the end of the arm (requires Arm Extend to be selected first — if not, gripper appears at arm stub on robot side).	Fingers close fully, hold for 0.5 seconds, then open. Arm joint LED turns green during close, off during open.	
Gripper Open (Release)	Same gripper finger assembly. Gripper Close and Gripper Open share the same physical component — both	Fingers open fully, hold for 0.5 seconds. Students see that Gripper Close and Gripper Open are two commands on the same	

	cards control the same fingers.	hardware — not two separate mechanisms.	
Arm Retract	No new component added — Arm Retract controls the same telescoping arm as Arm Extend. A retract indicator appears on the arm joint.	Arm extends briefly then folds flush against robot body. Students see that Extend and Retract are two commands on the same physical arm — both needed for the full pick-and-place cycle.	
Docking			
Dock Connector Engage 	Connector pin socket appears on robot base underside, rear-facing toward the dock.	Connector pin extends downward for 1 second, then retracts. Dock socket LED turns green during extension.	https://assetstore.unity.com/packages/3d/props/electronics/robotic-lawn-mower-set-334918
Additional False Actuators			
LED Body Flash	LED strip appears along the full perimeter of the robot body.	Strip pulses amber once, then blue once, then red once — showing the three available signal colors. Students see it is a feedback display, not a movement or cleaning mechanism.	
Bin Lid Open	Small wireless transmitter module appears on robot front face.	Transmitter blinks once. No other robot body change — this actuator sends a signal to the environment (the bin), not to any part of the robot itself.	
Speaker / Voice Output	Small speaker grille appears on robot head top surface.	Speaker plays a short two-tone chime on placement. Students can hear it is an audio output device — not a physical	

		movement or sensing capability.	
Arm Speed Control	Speed dial module appears on arm joint alongside the arm assembly.	Dial rotates slowly then quickly on placement. Students may select this believing it is needed for carrying fragile objects — they will discover in Step 4 that arm speed is already a programmable parameter in the block logic.	

3.5 - Completion Gate

Participants proceed to Step 4 only after all 10 slots are filled and both participants confirm. There is no correctness check at this stage.

Requirement	Detail
All 10 slots filled	The confirm button activates only when the slot counter reads 10 / 10. Partial completion is not accepted. Students cannot proceed with 9 slots — they must make a decision about every slot. The slot counter is always visible at the top of the center panel.
Joint confirmation	Both participants confirm together. The confirmation screen shows the final 10 selected actuators grouped by functional category. Either participant can return to the library from this screen to swap cards before committing.

3.6 - Data Capture

All Step 3 events are logged with PlayerID tags. Because placement is free and unrestricted, the log captures the full decision trajectory — not just the final state. The swap log is the richest signal: a swap indicates a participant actively reconsidered a choice, which combined with the post-session interview reveals the reasoning behind the revision.

Event	Data Captured (PlayerID tagged)	Maps to
Actuator card placed into slot	Actuator ID, PlayerID, timestamp, sequence order of all placements in the session.	RQ3 — contribution pattern; LO6 behavioral log.
Actuator card removed from slot	Actuator ID, PlayerID, timestamp.	RQ1 — evidence of revision; LO6.
Slot swap	When a removal is immediately followed by a new placement: actuator removed, actuator added, PlayerID, timestamp, time elapsed between removal and placement.	RQ1 — active reasoning about actuator necessity; LO6, LO8 precursor.
All 10 slots filled	Timestamp, final actuator set (all 10 IDs), PlayerID who placed the 10th card, total time spent in Step 3.	RQ2 — session pacing; baseline for Step 5 analysis.
Confirm and proceed	Final 10 actuator IDs, PlayerID who pressed confirm, timestamp. Post-session: final set compared against correct 10 to identify any false actuators selected.	RQ2 — primary behavioral baseline for Step 5 debug analysis.

Step 4 - Program Your Robot: Vibe Coding

PEAS: All four PEAS components
Learning Objectives: LO2 (construct full PEAS description), LO7 (trace perceive–decide–act cycle)
Competencies: C8 (Decision-Making), C17 (Programmability)
Purpose: Step 4 is where students translate their sensor and actuator selections from Steps 2 and 3 into executable decision logic. Using physical block manipulation, both participants construct a set of rules — one per task — that govern how their robot perceives its environment, evaluates conditions, and acts. Every rule follows a strict four-slot chain:

SENSE block	CONDITION block	ACTION block	CONTROL block
What can the robot perceive?	Under what condition?	What should the robot do?	How long / what priority?

All block logic is pre-coded. Students cannot write or modify code, they decide which blocks to combine. The code inside each block runs exactly as specified in the Block Library document. The six rules run in parallel during Step 5: whichever sensor fires first in a given tick triggers that rule's action. Rules do not execute in sequence; the robot is driven by the environment, not by a script.

4.1 - Block Library

Step 4 is where students build the decision logic for their cleaning robot by dragging blocks onto a canvas. Each rule on the canvas follows the same structure:

SENSE block	CONDITION block	ACTION block	CONTROL block
What can the robot perceive?	Under what condition?	What should the robot do?	How long / what priority?

All block logic is pre-coded by the development team and cannot be changed by students. Students decide which blocks to combine — the code inside each block runs exactly as specified below.

For developers: the "Returns" column in the SENSE table describes the exact value the sensor emits each tick. CONDITION blocks compare against these return values. ACTION blocks fire once per tick unless wrapped in a CONTROL block.

SENSE BLOCKS - Color: Blue

There are 15 sensor blocks in the library, one for each sensor that could be mounted on the robot. All 15 are visible to students at all times. Each SENSE block connects to one or more CONDITION blocks that use its return value.

Block ID	Name	What it detects	What it CANNOT detect	Returns	Body Module
SENSE_01	**Animal Detection Sensor**	Presence and position of tagged NPC_Animal objects within a 1.5 m radius	*Humans, furniture, walls. Only fires on the cat tag. Does not detect direction of travel.*	DETECTED / CLEAR	—

SENSE_02	**Battery Sensor**	Current battery level as a percentage	*Time remaining at current drain rate*	HIGH / MED / LOW	Chest bar
SENSE_03	**Destination Sensor**	Distance and direction to a tagged goal location: SINK, BIN, or DOCK	*Whether the destination is reachable; obstacles between robot and destination*	DIRECTION + DISTANCE	Head arrow
SENSE_04	**Distance Sensor**	Distance to nearest obstacle in 4 directions (front, back, left, right) up to 5 m	*What type the obstacle is (cat, furniture, wall, or person)*	NEAR / FAR / BLOCKED	Sonar rings
SENSE_05	**Floor Dirt Sensor**	Whether the current grid cell directly underfoot is tagged DIRTY	*Object type; cells not directly underfoot*	DIRTY / CLEAN	Undercarriage LED red
SENSE_06	**Floor Wet Sensor**	Whether the current grid cell directly underfoot is tagged WET	*Exact distances; dry dirt cells*	WET / DRY	Undercarriage LED blue
SENSE_07	**Floor Map Sensor**	State of every grid cell in the apartment: CLEAN / DIRTY / WET / VISITED / UNKNOWN	*Future state changes; moving objects*	FULL GRID MAP	—

SENSE_08	**Grip Sensor**	Whether the mechanical arm is currently holding an object successfully	*What the object is; grip pressure; whether it will drop if the robot moves fast*	GRIPPING / EMPTY	Arm joint LED green
SENSE_09	**Human Proximity Sensor**	Presence and distance of tagged PlayerAvatar objects within a 2 m radius	*The cat, furniture, or walls. Only fires on participant avatar tags.*	NEAR / CLEAR	Chest proximity ring blue
SENSE_10	**Light Level Sensor**	Ambient brightness in the current room in lux	*Movement, objects, obstacles — this sensor only reads light level*	DARK / DIM / BRIGHT	—
SENSE_11	**Object Weight Sensor**	Approximate weight of any object the arm is currently in contact with	*Object identity or fragility — only returns a weight reading*	WEIGHT (grams)	—
SENSE_12	**Session Timer Sensor**	How long the current task has been running, in seconds	*Whether the robot is stuck in a loop; environment state*	ELAPSED TIME (seconds)	Timer display
SENSE_13	**Sound Level Sensor**	Ambient noise level in the apartment in decibels	*Direction of sound; identity of sound source*	QUIET / LOUD	—

SENSE_14	**Stair Edge Detector**	Whether the robot is approaching a stair edge or floor drop-off	*General obstacles; walls; furniture; the cat*	EDGE DETECTED / CLEAR	—
SENSE_15	**Vision Sensor (Object Classifier)**	Type and position of tagged objects in a 120° forward-facing cone: CUP, FOOD, TRASH, BIN, SINK	*Exact distances; objects outside the cone; objects behind obstacles; untagged items*	OBJECT TYPE + POSITION	Blinking eyes on head

CONDITION BLOCKS - Color: Yellow

There are 20 condition blocks. Each one reads the return value from a specific SENSE block and evaluates whether a condition is true or false. A CONDITION block only fires if the SENSE block it depends on is present in the same rule chain. If a student places a CONDITION block without a compatible SENSE block, the canvas highlights the broken link.

Block ID	Display Name (IF...)	Requires SENSE	What this condition checks	Plain-language example
COND_01	**IF obstacle is NEAR**	SENSE_04	Distance sensor returns NEAR in any direction (front, back, left, or right)	*"If something is close to the robot, do something"*
COND_02	**IF obstacle is BLOCKED**	SENSE_04	Distance sensor returns BLOCKED — robot cannot move forward at all	*"If the robot is completely blocked, do something else"*
COND_03	**IF floor is DIRTY**	SENSE_05	Floor dirt sensor returns DIRTY on	*"If there is dirt here, clean it"*

			the current cell underfoot	
COND_04	**IF floor is WET**	SENSE_06	Floor wet sensor returns WET on the current cell underfoot	*"If there is a spill here, mop it"*
COND_05	**IF floor cell is UNVISITED**	SENSE_07	Floor map shows current cell as UNKNOWN (never visited this session)	*"If the robot has not been here yet, keep exploring"*
COND_06	**IF floor cell is CLEAN**	SENSE_07 or SENSE_05	Floor map or dirt sensor shows current cell is already CLEAN	*"If this spot is already done, move on"*
COND_07	**IF animal is DETECTED**	SENSE_01	Animal detection sensor returns DETECTED — cat is within 1.5 m	*"If the cat is nearby, react"*
COND_08	**IF battery is LOW**	SENSE_02	Battery sensor returns LOW (below 30%)	*"If battery is almost empty, go charge"*
COND_09	**IF battery is HIGH**	SENSE_02	Battery sensor returns HIGH (above 60%)	*"If battery is full, resume the task"*
COND_10	**IF destination is REACHED**	SENSE_03	Destination sensor returns distance = 0 — robot is at the target location	*"If the robot has arrived at the bin or dock, do the next step"*
COND_11	**IF destination is FAR**	SENSE_03	Destination sensor returns distance > 2 m	*"If the robot is still far from the target, keep moving"*

COND_12	**IF person is NEAR**	SENSE_09	Human proximity sensor returns NEAR — a participant avatar is within 2 m	*"If a person is close by, slow down or stop"*
COND_13	**IF person is CLEAR**	SENSE_09	Human proximity sensor returns CLEAR — no avatar within 2 m	*"If no one is nearby, resume normal speed"*
COND_14	**IF arm is GRIPPING**	SENSE_08	Grip sensor returns GRIPPING — arm has successfully grabbed an object	*"If the robot is holding something, carry it to the destination"*
COND_15	**IF arm is EMPTY**	SENSE_08	Grip sensor returns EMPTY — arm is not holding anything	*"If the robot dropped the object or missed, try again"*
COND_16	**IF object is TRASH**	SENSE_15	Vision sensor identifies the nearest object in the forward cone as TRASH	*"If the robot sees trash, pick it up"*
COND_17	**IF object is BELONGING**	SENSE_15	Vision sensor identifies the nearest object as CUP or FOOD (not trash)	*"If the robot sees something that belongs somewhere, sort it correctly"*
COND_18	**IF timer EXCEEDS limit**	SENSE_12	Session timer sensor returns elapsed time above a set threshold (e.g., 10 seconds)	*"If the robot has been waiting too long, give up*

				and try something else"
COND_19	**IF edge is DETECTED**	SENSE_14	Stair edge detector returns EDGE DETECTED — drop-off ahead	*"If the robot is about to fall off an edge, stop"*
COND_20	**IF weight is DETECTED**	SENSE_11	Object weight sensor returns a value above 0 — arm is touching an object	*"If the robot is touching something, check before grabbing"*

ACTION BLOCKS - Color: Green

These blocks define what the robot physically does

There are 23 action blocks, grouped by functional category. Each ACTION block maps directly to one actuator selected in Step 3. All 23 blocks are visible in the library at all times. Students must reason from their task requirements and their Step 3 actuator selections to determine which action blocks are relevant.

Block ID	Name	Requires Actuator	What the robot physically does	What it CANNOT do	Task(s)
LOCOMOTION					
ACT_01	**MOVE FORWARD**	*Wheel Drive*	Robot moves forward at current speed setting until a new command fires	*Cannot move sideways or diagonally*	1–6
ACT_02	**MOVE BACKWARD**	*Wheel Drive*	Robot moves backward at current speed setting	*Cannot strafe; turning not included*	1–6

ACT_03	**ROTATE LEFT 90°**	*Body Rotation*	Robot turns 90° left in place without moving forward or backward	*Cannot do partial turns; rotation is discrete (45°, 90°, 180°)*	1–6
ACT_04	**ROTATE RIGHT 90°**	*Body Rotation*	Robot turns 90° right in place without moving forward or backward	*Cannot do partial turns; rotation is discrete (45°, 90°, 180°)*	1–6
ACT_05	**ROTATE 180°**	*Body Rotation*	Robot turns around completely in place — reverses its facing direction	*Cannot do partial turns*	1–6
ACT_06	**EMERGENCY STOP**	*Emergency Stop*	All wheel motion halts immediately in the current tick. Overrides any active movement command.	*Does not reverse; does not move sideways; robot stays exactly where it stopped*	4, 6
ACT_07	**SET SPEED: SLOW**	*Wheel Drive*	Sets robot movement speed to slow mode — used when near people or carrying objects	*Does not move the robot by itself; only changes speed for the next movement command*	5

ACT_08	**SET SPEED: NORMAL**	*Wheel Drive*	Returns robot movement speed to normal after slow mode	*Does not move the robot by itself*	5
CLEANING					
ACT_09	**VACUUM ON**	*Vacuum Suction*	Activates suction on undercarriage. While active over a DIRTY cell, cell becomes CLEAN after 1 second of contact.	*Cannot clean WET cells — liquid damages the mechanism. Cannot pick up large objects.*	1
ACT_10	**VACUUM OFF**	*Vacuum Suction*	Deactivates suction — used when robot is moving between dirty areas to save power	*Does not move the robot*	1
ACT_11	**MOP DEPLOY**	*Mop Pad Deploy/Retract*	Lowers mop attachment from undercarriage. While deployed over a WET cell, cell becomes CLEAN after 2 seconds of contact.	*Cannot be active simultaneously with vacuum suction. Cannot clean DIRTY (dry) cells.*	3

ACT_12	**MOP RETRACT**	*Mop Pad Deploy/Retract*	Raises mop pad back to retracted position after cleaning is complete	*Does not clean anything; only retracts the physical pad*	3
MECHANICAL ARM					
ACT_13	**ARM EXTEND**	*Arm Extend*	Telescopes the arm outward up to 0.6 m toward a tagged object. Arm aligns to object position automatically.	*Cannot reach objects further than 0.6 m; cannot extend through walls or furniture*	2
ACT_14	**GRIPPER CLOSE (Grab)**	*Gripper Close*	Closes gripper fingers around the nearest tagged object within reach. On success, grip sensor returns GRIPPING.	*Cannot grab objects the arm has not extended toward; cannot hold two objects at once*	2
ACT_15	**GRIPPER OPEN (Release)**	*Gripper Open*	Opens gripper fingers, releasing the held object. Object drops to the surface below the arm's current position.	*Cannot place an object precisely — drops at arm's current position and falls to gravity*	2
ACT_16	**ARM RETRACT**	*Arm Retract*	Pulls the arm back to the folded home	*Cannot retract while the gripper is*	2

			position flush against the robot chassis after use.	*still closed around an object*	
DOCKING					
ACT_17	**DOCK CONNECTOR ENGAGE**	*Dock Connector Engage*	When within 0.2 m of the charging dock, deploys a physical connector pin into the dock socket. Charging begins.	*Cannot engage from further than 0.2 m; robot must be precisely aligned with the dock socket*	6
FEEDBACK & ENVIRONMENT					
ACT_18	**LED FLASH: RED**	*LED Body Flash*	Pulses the LED strip red — conventional signal for an error or blocked state	*Cannot display text; only fixed color signals*	1–6
ACT_19	**LED FLASH: AMBER**	*LED Body Flash*	Pulses the LED strip amber — caution signal (e.g., near person, carrying object)	*Cannot display text; only fixed color signals*	2, 5
ACT_20	**LED FLASH: BLUE**	*LED Body Flash*	Pulses the LED strip blue — active signal (e.g., mopping, carrying)	*Cannot display text; only fixed color signals*	2, 3

ACT_21	**BIN LID OPEN**	*Bin Lid Open*	Sends a wireless signal to the smart bin in the environment. The bin lid opens automatically, ready to receive a deposited object.	*Cannot open the sink drain or any standard open drop zone. Requires a smart bin in the environment.*	2
ACT_22	**PLAY SOUND: BEEP**	*Speaker/Voice Output*	Emits a short beep tone — used to signal task completion or a state change	*Cannot produce arbitrary speech; limited to a fixed library of tones and chimes*	1–6
ACT_23	**PLAY SOUND: CHIME**	*Speaker/Voice Output*	Emits a chime tone — used to signal battery warning or successful docking	*Cannot produce arbitrary speech; limited to a fixed library of tones and chimes*	6

CONTROL BLOCKS - Color: Grey

These blocks modify how and when a rule executes

There are 6 control blocks. Unlike SENSE, CONDITION, and ACTION blocks, CONTROL blocks do not represent physical robot components — they represent execution logic. They are always available and do not depend on any sensor or actuator selection. CONTROL blocks are the fourth (optional) slot in a rule chain and connect to the right of an ACTION block.

Block ID	Name	What it controls	Plain-language example
CTRL_01	**REPEAT UNTIL done**	Loops the attached ACTION block repeatedly until the CONDITION in the same rule is no longer true. Stops automatically when the condition clears.	*"Keep vacuuming this cell UNTIL the floor dirt sensor reads CLEAN"*
CTRL_02	**REPEAT WHILE true**	Loops the attached ACTION block for as long as the CONDITION remains true. Fires every tick the condition holds.	*"Keep moving backward WHILE obstacle is NEAR"*
CTRL_03	**WAIT 2 seconds**	Pauses execution of this rule for 2 seconds before the ACTION fires. Robot holds its current state during the wait.	*"Wait 2 seconds after the cat is detected — then reroute. Prevents jittery stop-start behavior."*
CTRL_04	**THEN (sequence)**	Connects two ACTION blocks in sequence: ACTION A fires first, then ACTION B fires immediately after. Used when one task physically requires two steps.	*"ARM EXTEND → THEN → GRIPPER CLOSE: extend the arm, then immediately grab the object"*
CTRL_05	**PRIORITY: HIGH**	Marks this rule as high priority. If two rules could fire at the same time, the HIGH priority rule wins and the other is suppressed for that tick.	*"EMERGENCY STOP rule must always win over the MOVE FORWARD rule when the cat appears"*

CTRL_06	**PRIORITY: NORMAL**	Default priority. Used explicitly to mark a rule as lower priority than a HIGH rule. No effect unless a HIGH priority rule is also present.	*"Normal movement rules run at NORMAL priority so safety rules can always override them"*

Complete Rule Examples

The table below shows one correct rule for each of the six tasks. These are not the only valid rules — they illustrate the intended SENSE → CONDITION → ACTION → CONTROL pattern. Students may construct different valid rules during the session. These examples are provided for developer testing and for the researcher to use in post-session interviews.

Task	SENSE block	CONDITION block	ACTION block	CONTROL block (optional)	Why this rule works
Task 1 — Vacuum floor	SENSE_05 Floor Dirt Sensor	COND_03 IF floor is DIRTY	ACT_09 VACUUM ON	CTRL_01 REPEAT UNTIL done	*Robot checks the cell underfoot, vacuums while it is dirty, stops when clean — then needs a movement rule to drive to the next cell*
Task 2 — Pick up object	SENSE_15 Vision Sensor	COND_16 IF object is TRASH	ACT_13 ARM EXTEND → (CTRL_04 THEN) → ACT_14 GRIPPER CLOSE	CTRL_04 THEN (sequence)	*Robot sees trash, extends arm, then immediately closes gripper — two physical steps chained in sequence*

Task 3 — Clean spill	SENSE_06 Floor Wet Sensor	COND_04 IF floor is WET	ACT_11 MOP DEPLOY	CTRL_01 REPEAT UNTIL done	*Robot deploys mop while cell is wet; retracts automatically when cell becomes CLEAN*
Task 4 — Avoid cat	SENSE_01 Animal Detection	COND_07 IF animal is DETECTED	ACT_06 EMERGENCY STOP	CTRL_05 PRIORITY: HIGH	*Emergency stop must always override movement rules when the cat appears — high priority ensures it wins*
Task 5 — Slow near people	SENSE_09 Human Proximity	COND_12 IF person is NEAR	ACT_07 SET SPEED: SLOW	CTRL_02 REPEAT WHILE true	*Robot stays in slow mode for as long as the person remains within 2 m — returns to normal speed when they leave*
Task 6 — Return to dock	SENSE_02 Battery Sensor	COND_08 IF battery is LOW	ACT_06 EMERGENCY STOP → (CTRL_04 THEN) →	CTRL_05 PRIORITY: HIGH	*Battery low triggers high-priority interruption of current*

			ACT_01 MOVE FORWARD (toward dock)		*task, then robot navigates to dock*

Implementation Notes

6.1 — Rule validation logic

A rule chain is valid and executable when it contains at minimum one SENSE block and one ACTION block. A CONDITION block without a compatible SENSE block in the same chain must be flagged with a yellow broken-link indicator on the canvas — but not removed. The student can fix it by adding the correct SENSE block. A CONTROL block without an ACTION block should be flagged similarly.

6.2 — Tick-based execution

The robot simulation runs on a fixed tick (recommended: 10 ticks per second). Each tick, the robot evaluates all active rules in the SharedRuleSet. Rules without a CONTROL block fire once per tick while their condition is true. REPEAT UNTIL and REPEAT WHILE fire every tick. WAIT pauses the rule for a fixed number of ticks (2 seconds = 20 ticks at 10 Hz). PRIORITY resolves conflicts: if two rules fire in the same tick, the HIGH priority rule wins and the other is suppressed for that tick only.

6.3 — PlayerID tagging

Every block placement, removal, and swap event must be logged with the PlayerID of the participant who triggered it, a timestamp, and the sequence order within the session. This is required for RQ3 contribution analysis. The PlacedByPlayerID field in the RuleData struct (defined in Part A of the design flow document) stores this information at the rule level. Individual placement events are stored separately in the session event log.

6.4 — Broken link visual feedback

When a CONDITION block is placed without a compatible SENSE block: the CONDITION block glows amber on the canvas and a floating label reads 'This condition needs a sensor — add the matching SENSE block to complete this rule.' When a student adds the correct SENSE block, the amber glow clears. This feedback must be visible to both participants simultaneously via Fusion sync.

6.5 — Starter template

The canvas opens with one pre-loaded rule: SENSE_04 (Distance Sensor) → COND_01 (IF obstacle NEAR) → ACT_06 (EMERGENCY STOP) → CTRL_05 (PRIORITY: HIGH). This rule is intentionally incomplete for the full task set — the robot will stop near obstacles but cannot reroute. This seeds the debug loop in Step 5 without requiring students to build from a blank canvas.

6.6 — Right-wall robot preview sync

The robot preview on the right wall must update within 0.5 seconds of any block change on the canvas. This is handled by the OnRuleSetChanged callback on the SharedRuleSet NetworkArray (see Part A, section A3 of the design flow document). The preview runs the

robot against a miniature version of the apartment using the current rule set. The perceive–decide–act trace panel overlays which block fired during the most recent tick.

4.2 - Logic Room Layout

FACING WALL — Block Canvas	RIGHT WALL — Robot Preview	LEFT WALL — Block Library	FLOOR — PEAS Map
The main work surface. Six task zones are laid out vertically, one per task. Each zone is colour-coded and labelled with the task name. Inside each zone there are four slots arranged left-to-right: **[SENSE]** → **[COND]** → **[ACT]** → **[CTRL]** *(optional)* Slots are physical 3D snap-points in VR. A slot glows white when empty and ready to receive a block. After a block is snapped in, the next slot lights up automatically.	A miniature apartment running the robot simulation in real time. Synced across both clients via OnRuleSetChanged. Updates within 0.5 s of any block placement or removal. Below the simulation: the Perceive–Decide–Act trace panel. Shows which block fired during the most recent tick in three colour-coded lines: **SENSING** which sensor triggered **DECIDING** which condition evaluated true **ACTING** which action the robot executed Either participant may tap any placed block on the facing wall to trigger a trace highlight — the block glows and the trace panel highlights the corresponding line simultaneously on both clients.	All blocks available in the session, organised into four colour-coded shelves: **SENSE blocks** (15 blocks) **CONDITION blocks** (20 blocks) **ACTION blocks** (23 blocks) **CONTROL blocks** (6 blocks) Each block card shows: Block ID, plain-language name, and a one-line description. Either participant can walk to the library, pick up any block, and carry it to the canvas. Block weight is zero — picking up and carrying is effortless in VR.	The floor is divided into four colour-coded zones matching the four PEAS components: **P** Performance — yellow zone **E** Environment — grey zone **A** Actuators — green zone **S** Sensors — blue zone When a block is snapped into a slot on the canvas, the corresponding floor zone briefly glows — connecting the block type to its PEAS component. This is a passive, ambient reminder; it does not interrupt the workflow.

The Contribution Tracker is a small shared panel mounted at eye level between the canvas and the library. It shows in real time: blocks placed by Player A this session / blocks placed by Player B this session, broken down by task zone. This panel is used post-session for RQ3 contribution pattern analysis. It is visible to both participants during the session — its presence is not hidden, but it is not foregrounded as a competitive scoreboard.

4.3 - Block Chain Interaction Logic

Each task zone on the facing wall contains four slots arranged left to right. Slots activate in sequence. A student cannot skip a slot or fill slots out of order. The following table defines what happens at each slot.

Slot	Rule	What happens in VR
Slot 1 — SENSE	Must be filled first. No other slot is active until a SENSE block is placed. The SENSE slot is always lit white at the start of every task zone.	The student walks to the left library shelf (blue), picks up a SENSE block, carries it to the task zone, and snaps it into Slot 1. The block locks in with a click sound. Slot 2 (CONDITION) immediately lights up. The right-wall robot preview does not update yet — a SENSE block alone does not produce robot behaviour.
Slot 2 — CONDITION	Becomes available after Slot 1 is filled. Only CONDITION blocks that are compatible with the placed SENSE block can snap into Slot 2. If a student tries to snap an incompatible CONDITION block, it bounces back to their hand with a soft audio cue and a floating label: 'This condition needs a different sensor.' The label fades after 2 seconds.	The student walks to the yellow shelf, reads CONDITION block descriptions, selects the one that matches their intent, and snaps it into Slot 2. Slot 3 (ACTION) lights up. Still no robot preview update — a SENSE + CONDITION chain without an ACTION has no executable behaviour.

Slot 3 — ACTION	Becomes available after Slot 2 is filled. Any ACTION block can snap into Slot 3 regardless of which SENSE or CONDITION is in the chain — the student is responsible for choosing an action that makes sense for the condition. This is a deliberate design choice: incorrect action selection is a learning opportunity, not a gate. When Slot 3 is filled, the rule is complete and executable. The right-wall robot preview updates immediately.	The student walks to the green shelf, picks an ACTION block, and snaps it into Slot 3. The robot preview on the right wall updates within 0.5 s — the robot begins reacting to the new rule in the miniature simulation. The trace panel activates and shows SENSING / DECIDING / ACTING for that rule on the next triggered tick. Slot 4 (CONTROL) lights up with a dimmed optional indicator.
Slot 4 — CONTROL	Available after Slot 3 is filled but labelled 'optional' in dim text above the slot. If left empty, the rule executes once per tick while its condition is true — this is the default behaviour. A CONTROL block modifies that default: REPEAT UNTIL, REPEAT WHILE, WAIT, THEN, or PRIORITY.	Students may walk to the grey shelf and add a CONTROL block if they want to modify how the rule executes. Adding or removing a CONTROL block updates the robot preview immediately. Either participant can add or remove the CONTROL block at any time — it does not lock the chain.

One important clarification about removing blocks

When a student removes a block from Slot 1 or Slot 2, the blocks in downstream slots of the same chain are NOT automatically removed. They remain in their slots but become visually dimmed, and the task zone header shows a warning: 'Incomplete rule — Slot 1 is empty.' The student must either replace the missing block or manually remove the downstream blocks.

4.4 - SENSE → CONDITION Compatibility

Not all CONDITION blocks are compatible with all SENSE blocks. A CONDITION block can only snap into Slot 2 if it reads the same return value that the Slot 1 SENSE block emits. The table below specifies which CONDITION blocks are compatible with each SENSE block, and why. This is the rule the system uses to validate snaps and bounce incompatible blocks back to the student's hand.

SENSE block placed	Compatible CONDITION blocks	Why only these
SENSE_01 Animal Detection	COND_07 IF animal DETECTED	This sensor only returns DETECTED or CLEAR — there is only one meaningful condition to write against it.
SENSE_02 Battery Sensor	COND_08 IF battery LOW COND_09 IF battery HIGH	The sensor returns HIGH / MED / LOW. Two threshold conditions cover the relevant design decisions: go charge, or resume task.
SENSE_03 Destination Sensor	COND_10 IF destination REACHED COND_11 IF destination FAR	The sensor returns direction + distance. Students can check whether the robot has arrived or is still approaching.
SENSE_04 Distance Sensor	COND_01 IF obstacle NEAR COND_02 IF obstacle BLOCKED	The sensor returns NEAR / FAR / BLOCKED in four directions. Two conditions cover the key navigational decisions: slow/stop, or completely reroute.
SENSE_05 Floor Dirt Sensor	COND_03 IF floor DIRTY COND_06 IF floor cell CLEAN	The sensor returns DIRTY or CLEAN for the current cell. Both conditions are useful: trigger

		cleaning, or confirm and move on.
SENSE_06 Floor Wet Sensor	COND_04 IF floor WET COND_06 IF floor cell CLEAN	The sensor returns WET or DRY. COND_06 is shared with SENSE_05 — both sensors can confirm a cell is clean.
SENSE_07 Floor Map Sensor	COND_05 IF floor cell UNVISITED COND_06 IF floor cell CLEAN	The sensor returns the full grid state. Two coverage conditions: explore unvisited cells, or skip already-clean cells.
SENSE_08 Grip Sensor	COND_14 IF arm GRIPPING COND_15 IF arm EMPTY	The sensor returns GRIPPING or EMPTY. Both conditions are needed for the full pick-and-place cycle.
SENSE_09 Human Proximity	COND_12 IF person NEAR COND_13 IF person CLEAR	The sensor returns NEAR or CLEAR. Both conditions are needed: slow down when near, resume when clear.
SENSE_10 Light Level	COND_20 IF weight DETECTED (none — no compatible condition)	This sensor has no compatible condition for the six tasks. It appears in the library as a plausible-but-irrelevant option. Students who select it will find no matching CONDITION block snaps successfully — a productive discovery moment.
SENSE_11 Object Weight	COND_20 IF weight DETECTED	The sensor returns a weight value. One condition: confirm the arm is touching something before attempting to grab.

SENSE_12 Session Timer	COND_18 IF timer EXCEEDS limit	The sensor returns elapsed time. One condition: escape a stuck state after a timeout threshold.
SENSE_13 Sound Level	(none — no compatible condition)	This sensor has no compatible condition for the six tasks. Same discovery mechanic as SENSE_10.
SENSE_14 Stair Edge Detector	COND_19 IF edge DETECTED	The sensor returns EDGE DETECTED or CLEAR. One condition: stop before a drop-off.
SENSE_15 Vision Sensor	COND_16 IF object TRASH COND_17 IF object BELONGING	The sensor classifies objects in the forward cone. Two conditions cover the sorting task: handle trash vs. handle belongings differently.

Developer note — Compatibility enforcement

Compatibility is enforced at the slot level in Unity. Each CONDITION block prefab carries a CompatibleSenseIDs string array. When a student attempts to snap a CONDITION block into Slot 2, the slot checks whether the SENSE block currently in Slot 1 of the same chain has its ID in the CompatibleSenseIDs array. If yes: snap succeeds. If no: snap fails silently with bounce animation and audio cue. No error message lists which sensor is required — the student must read the block descriptions to work it out. SENSE_10 (Light) and SENSE_13 (Sound) have no compatible CONDITION blocks; their CompatibleSenseIDs array is empty. Any attempt to snap a CONDITION into Slot 2 after placing these sensors will always bounce.

4.5 - Six Task Zones on the Facing Wall Canvas

The facing wall canvas is divided into six task zones, arranged top to bottom in the order the tasks were introduced in Step 1. Each zone contains a task card plus four slots. Task zones are organizational — they help students think about which rule serves which task — but they do not enforce task-to-rule correctness. A student may technically place any rule in any task zone. Task correctness is evaluated in Step 5 by the performance dashboard, not by the canvas.

Task Zone	What the task card shows inside each zone	Minimum rule(s) needed for this task to work in Step 5
Task 1 Vacuum the floor	Task name + one-line description: 'Move across the floor and clean dirty cells.' Below the description: the performance dimension this task contributes to — Coverage.	*One rule connecting a floor-state sensor to a vacuum action. Without a movement rule (MOVE FORWARD + ROTATE) added separately, the robot will vacuum the starting cell and stop.*
Task 2 Pick up and sort objects	Task name + description: 'Navigate to scattered objects, pick them up, carry them to the correct destination.' Performance dimension: Coverage + Hazard Response.	*Minimum three rules: one to detect and grab the object, one to carry and release at the correct bin (TRASH vs. BELONGING branch), one to retract the arm after release.*
Task 3 Detect and clean spills	Task name + description: 'Find wet cells and deploy the mop until the cell is clean.' Performance dimension: Hazard Response.	*One rule: wet sensor → IF WET → MOP DEPLOY + REPEAT UNTIL done. Without a movement rule, the robot only cleans spills it starts on top of.*
Task 4 Detect and avoid the cat	Task name + description: 'Stop immediately when the cat appears in the robot's path, then reroute.' Performance dimension: Safety.	*Two rules: one HIGH PRIORITY emergency stop when the animal sensor fires, one rotation rule to reroute after the stop.*

Task 5 Slow down near participants	Task name + description: 'Reduce movement speed when a participant is within 2 m. Resume normal speed when they leave.' Performance dimension: Safety.	*Two rules: SET SPEED SLOW when person NEAR (REPEAT WHILE), SET SPEED NORMAL when person CLEAR.*
Task 6 Return to dock when battery low	Task name + description: 'Interrupt the current task when battery is LOW and navigate back to the charging dock.' Performance dimension: Coverage (sustained over time).	*Two rules: HIGH PRIORITY emergency stop + navigation toward dock when battery LOW; DOCK CONNECTOR ENGAGE when destination REACHED.*

Each task zone also shows a small PEAS component indicator in the top-right corner of the zone header — a coloured letter (P / E / A / S) indicating which PEAS component is most directly exercised by that task's rule design. This is a passive label, not a gate.

4.6 - System Feedback States

The following table defines all feedback states the system must produce during Step 4. Each state is triggered by a specific student action and serves a specific learning purpose.

Situation	What the student sees / hears	Learning purpose
Incompatible CONDITION block snapped into Slot 2	Block bounces back to student's hand. Soft audio cue. Floating label for 2 seconds: 'This condition needs a different sensor.' Label fades automatically.	*Forces the student to think about why a specific condition requires a specific sensor — the connection between perception and decision is not arbitrary.*
Slot 3 (ACTION) filled - rule becomes executable	Robot preview on right wall updates immediately. Trace panel activates. The floor zone for the relevant PEAS component glows briefly.	*The moment the rule becomes complete is made visceral — the robot starts responding. This is the core embodied authorship moment for C17 (Programmability).*
Robot preview shows unexpected behavior	The trace panel on the right wall shows which block fired. Either participant can tap the block on the canvas wall to trigger a full trace highlight — block glows, trace panel highlights the corresponding line.	*Connects observed behaviour back to authored specification. Directly supports LO7 (trace perceive–decide–act cycle) and seeds the debug reasoning needed in Step 5.*
Task zone has no rule yet (all slots empty)	Task zone header pulses dim grey. Floating prompt above the zone: 'This task has no rule yet — your robot will not attempt it.'	*Makes the absence of a rule visible as a design gap, not a neutral state. Students must actively decide to leave a task without a rule — it cannot happen by accident.*

Starter template rule fires during preview	SENSE_04 → COND_01 → ACT_06 fires when the preview robot approaches an obstacle. Robot stops. Trace panel shows the rule. The robot does not reroute — it stays stopped.	*Deliberately incomplete: the robot stops but cannot navigate around the obstacle. This is the seed for Step 5 debug — students will see the gap between 'stopped' and 'rerouted' in the test run.*
Both participants place blocks simultaneously	First grab wins (Fusion StateAuthority transfer). Second participant's grab fails silently — their hand passes through the block for 0.3 s. The block stays with the first grabber.	*Simultaneous conflict is resolved automatically with no interruption to the session. The 0.3 s pass-through is brief enough not to disrupt flow but distinct enough to signal that the block is taken.*
Completion gate check	System counts complete rules (minimum Slot 1 + Slot 3 filled) across all six task zones. Floating prompt appears: 'Your robot can handle [N] of the 6 tasks you selected. Add more rules before the test run, or continue.' Either participant confirms.	*Ensures students consciously know which tasks their robot can and cannot attempt before entering Step 5. The prompt does not block proceeding — it informs.*

4.7 - Collaborative Interaction (Symmetric Model)

Both participants have identical affordances throughout Step 4. Neither participant is assigned a role. Either participant may interact with any element at any time. Coordination emerges through voice — pairs naturally divide attention between library exploration and canvas building, or between block placement and preview watching, without being prompted to do so. The following table defines the symmetric access model for Step 4.

Action	Either participant can…	Both participants see…
Pick up a block from library	Walk to any shelf, grab any block at any time — no role restriction	The block lifts off the shelf and moves with the grabber's hand, visible to both via NetworkTransform sync
Snap a block into a canvas slot	Place any compatible block into any open slot in any task zone	Block locks into the slot with a click. Next slot lights up on both clients simultaneously via OnRuleSetChanged
Remove a block from a slot	Grab any placed block and pull it out of the slot at any time — including blocks placed by the other participant	Block returns to floating state in the grabber's hand. Slot reopens (glows white). Robot preview updates within 0.5 s. Downstream slots in the same chain do not clear automatically — only the removed block's slot reopens
Tap a block for trace highlight	Physically tap any placed block on the canvas wall using the controller	The tapped block glows on both clients. The right-wall trace panel highlights the corresponding SENSING / DECIDING / ACTING line simultaneously

Read block descriptions in library	Walk to any shelf and inspect block cards — no time limit, no penalty	Only the inspecting participant's hand highlights the card on hover. The other participant sees their partner's avatar standing at the shelf but does not see the card text unless they approach
Confirm completion gate	Press the confirm button that appears when the gate prompt is shown	Both participants see the gate prompt. Either can confirm or dismiss it. Confirmation is logged with PlayerID.

The natural division of labour observed in pilot testing (one participant focuses on the canvas, the other monitors the preview and narrates) is a primary data source for RQ3. Because no role is enforced, any division that emerges reflects the pair's own coordination strategy — which is exactly what the study aims to document.

4.8 - Data Capture

All Step 4 events are logged with PlayerID tags. Because block placement is free and unrestricted within the slot ordering rules, the log captures the full decision trajectory — not just the final rule state. The swap log is the richest signal: a swap indicates a student actively reconsidered a design choice, which combined with the post-session interview reveals the reasoning behind the revision.

Event	Data captured (PlayerID tagged)	Maps to	Measure type
Block placed into slot	Block ID, slot position (task zone + slot number), PlayerID, timestamp, sequence order in session	RQ3 — contribution pattern; LO7 behavioural log	*Behavioural log*

Block removed from slot	Block ID, slot position, PlayerID, timestamp	RQ1 — evidence of revision; LO7	*Behavioural log*
Block swap (remove then place within 10 s)	Block removed ID, block added ID, slot position, PlayerID, timestamp, elapsed time between removal and placement	RQ1 — active reasoning; LO7, LO8 precursor	*Behavioural log*
Incompatible snap attempt	Block ID attempted, slot position, PlayerID, timestamp, SENSE block currently in Slot 1 of that chain	RQ2 — sensor-condition mapping errors; C15	*Behavioural log*
Trace highlight triggered	Block ID tapped, task zone, PlayerID, timestamp, which trace line was highlighted (SENSING/DECIDING/ACTING)	RQ2 — LO7 engagement	*Behavioural log*
Robot preview rule fired (per tick)	Rule ID, task zone, SENSE+COND+ACT+CTRL block IDs, timestamp of tick	RQ1 — perceive–decide–act cycle tracing	*System log*
Completion gate confirmed	N rules complete, N tasks covered, PlayerID who confirmed, timestamp, total time in Step 4	RQ2 — session pacing	*Behavioural log*
Contribution tracker state (end of Step 4)	Blocks placed by Player A total / Player B total, per task zone breakdown	RQ3 — primary contribution measure	*Behavioural log*

Step 5 - Test, Debug, and Reflect

PEAS: All four PEAS components
Learning Objectives: LO3 (evaluate behavior), LO7 (trace cycle), LO8 (revise specification)
Competencies: C1 Recognizing AI, C8 Decision-Making, C14 Action & Reaction, C17 Programmability
Purpose: Step 5 is the highest-leverage learning moment. The pair runs the robot, documents failures, enters the Debug Room, revises, and iterates. All five target competencies converge here. In the symmetric model, both participants share all affordances throughout the test run and debug cycle — verbal agreement on diagnosis before execution is a natural expectation that emerges from the task, not an enforced gate.

5.1 - Test Run (Full Apartment)

Interface Element	Content and Behavior
Apartment view	Full apartment. Robot runs autonomously. Both participants can walk anywhere — they are co-present with the running robot.
Performance dashboard	Shared panel on apartment wall. Both participants see the same live scores on all four dimensions.
Perceive–decide–act trace panel	Either participant can trigger by pointing at robot and pressing controller. Both see the trace panel appear simultaneously.
Observation log	Both participants can stamp ✓ / ✗ /? on any observed behavior. Each stamp is tagged with PlayerID. Both participants see all stamps on the shared log.
Robot POV button	Either participant can enter Robot POV during the test run to check sensor coverage from the robot's perspective.

🔧 **DRONE PROJECT REFERENCE**

Feasibility: Direct reuse — drone ScoreBoard + LearnerActivityTracker pattern

Module to borrow: ScoreBoardManager + LearnerActivityTracker.Progress (networked float)

How to adapt: The drone's performance dashboard listens to onLearnerActivityTrackerUpdate events and updates progress bars for all participants. Your dashboard listens to OnRuleSetChanged and OnRobotActionCompleted, recomputes four dimension scores, and updates shared NetworkDictionary<int, float> DimensionScores. Both clients read this and display the same values.

Code note: [Networked] public NetworkDictionary<int, float> DimensionScores { get; } // 0=Coverage, 1=Safety, 2=Speed, 3=Hazard

5.2 - Debug Room (Collaborative, Symmetric)

Interface Element	Content and Behavior
Facing wall — block canvas	Same as Step 4. Either participant can grab and modify any block. Either participant can tap for trace highlights.
Right wall — robot preview	Both participants see identical preview. Updates on all clients via OnRuleSetChanged.
Left wall — observation log	All stamps from both participants displayed. ✗ and ? entries highlighted amber. Either participant can tap an entry to trigger failure playback.
Floor — PEAS map	Same color zones. When a debug move is needed in an earlier step, the relevant floor zone pulses — both participants see it simultaneously.

5.3 - The Three Debug Moves (Symmetric Version)

Debug Move	Either Participant Can	Coordination Expected	Competency / LO
Move 1 Swap / modify a block	Pick up a highlighted block, discuss replacement with partner, grab new block from shelf	Both discuss diagnosis before either touches the canvas — the 2-second delay prompt ('What caused this? Discuss before changing anything.') appears after any log entry is tapped	C17: behavior is specification. LO8.
Move 2 Add missing rule	Take blank IF–THEN frame from shelf, use vibe coding, connect to canvas	Either participant may suggest the needed sensor — the one who did Robot POV may have insight into the gap. Either acts on the suggestion.	C17 + C8: bounded world. LO8, LO2.
Move 3 Return to earlier step	Walk toward Debug Room door; follow floor arrow to Step 2 or Step 3; return and report changes to partner	Both discuss which step to return to. Either participant executes the return. Both see the floor zone pulse indicating the relevant PEAS component.	C14 + C1: systemic design. LO8, LO3.

Verbal Agreement Protocol (Symmetric)

Before either participant executes any debug move, the system displays a 2-second prompt (visible to both): 'What caused this? Discuss before changing anything.'

This prompt is not a gate — it disappears after 2 seconds and does not lock interaction. But it creates a structured moment for verbal coordination. Either participant can proceed after the prompt fades. The verbal discussion that follows is a primary data source for the post-session interview and for analyzing collaborative decision-making patterns in the symmetric model.

5.4 - PEAS Reflection Prompt (Spatially Embedded)

When a performance score drops, the thought-bubble prompt appears near the failing robot in 3D space. Both participants see it simultaneously — it is a shared spatial object, not a per-player UI element. Either participant may select the answer.

PEAS Reflection Prompt Text

'Your robot just [behavior]. This affected your [dimension] score.'

'Before you change anything: is your robot behaving irrationally — or rationally with a wrong specification?'

Either participant selects on behalf of the pair:

- My sensor doesn't detect what I expected
- My decision rule fires at the wrong time
- My actuator can't do what my performance measure requires
- My performance measure weights are set incorrectly

5.5 - Closing C1 Reflection (Both Participants — Individual)

At the end of the final iteration, before the summary screen, both participants respond to the closing reflection prompt separately. This is the second moment (after 1A) where individual responses are required — both answer simultaneously but independently. This is a measurement task, not a design task.

Closing Reflection Prompt

'Think back to the three objects you saw at the beginning — the robotic vacuum, the push vacuum, and the smart thermostat.'

'Your robot is one more AI agent in that apartment now.'

'What can you change about it? What does that tell you about what makes it an AI agent?'

Both participants respond via voice or keyboard. Responses are individually transcribed and tagged with PlayerID.

5.6 - PEAS Summary Screen (Shared)

Panel	Content
Purpose	Verbatim purpose statement from Step 1
Performance measure	Four slider values with ranked priority list
Environment	Dimension descriptors from Step 2A
Sensors	Selected sensors and SENSE blocks unlocked
Actuators	Selected actuators with Step 3 justification notes
Tasks completed	X / Y tasks at threshold on final iteration
Performance scores	Breakdown by dimension; delta from first test run to final
Debug iterations	Total iterations; breakdown of Debug Moves 1/2/3; blocks changed per session
Contribution breakdown	Blocks placed by Player A / blocks placed by Player B. Log stamps by Player A / by Player B. Debug moves initiated by Player A / by Player B.
Most revised component	Which PEAS component (P/E/S/A) the pair returned to most often

Part C - Data Capture Summary (Symmetric)

In the symmetric model, all behavioral logs include PlayerID tags. Because there are no enforced roles, the analysis focus shifts from role comparison (Builder vs. Observer) to contribution pattern analysis: did pairs naturally divide attention, did one participant dominate interaction, did verbal coordination before block placement predict better debug moves? Post-session individual knowledge assessments and surveys remain unchanged.

System Event	Data Captured (PlayerID tagged)	Maps to RQ	Measure Type
Step 1A — C1 baseline	Both players' selections, independently logged	RQ2 — C1 pre (individual)	Knowledge assessment (pre)
Step 1B — Task selection	Tasks selected with PlayerID; timestamp; sequence of selections	RQ1 — PEAS P understanding	Behavioral log
Step 1C — Slider config	Slider values; which player moved each slider; conflicts; confirm timestamp	RQ2 — LO3, C8	Behavioral log
Step 2B — Sensor selection	Sensors selected with PlayerID; recommendation flags noted/ignored	RQ2 — LO5, C15	Behavioral log
Step 2C — Robot POV	Which player entered POV; duration; head movement range; whether both entered	RQ1 — VR mechanism for C15	Sensor log (VR only)
Step 3 — Actuator selection	Actuators selected with PlayerID; alignment check outcomes	RQ2 — LO6, C14	Behavioral log
Step 4 — Block building	All placements/removals/swaps with PlayerID; vibe coding calls + LLM mappings; grab conflict events	RQ1 — C8, C17 authorship	Behavioral log
Step 4 — Trace highlights	Which player tapped which block; frequency; dwell time	RQ2 — LO7	Behavioral log
Step 5 — Test run scores	Dashboard values at end of each test run (shared panel — one log entry)	RQ2 — LO3	Performance measure
Step 5 — Observation log	All ✓ / ✗ /? stamps with PlayerID; timestamp; sequence of stamping	RQ1 — failure identification	Behavioral log
Step 5 — Debug moves	Move type 1/2/3 with PlayerID; who initiated; PEAS steps returned to;	RQ1 + RQ2 — LO8	Behavioral log

	blocks changed; verbal agreement delay		
Step 5 — PEAS prompt response	Which option selected; PlayerID; time to select after prompt appeared	RQ2 — C8, LO8	Behavioral log
Step 5E — Closing reflection	Both players' responses individually transcribed with PlayerID	RQ2 — C1 post (individual)	Qualitative (per-player)
Post-session knowledge test	Both players complete individually (separated after VR session)	RQ2 — primary outcomes (individual)	Knowledge assessment (post)
Post-session survey	Both players complete individually: engagement, confidence, satisfaction, interest	RQ2 — learning measures (individual)	Survey (Likert, per-player)
Post-session interview	Pair interviewed together first; then each individually	RQ1 + RQ2	Semi-structured interview

Contribution Analysis in the Symmetric Model

In v4, role assignment (Builder/Observer) provided a structural basis for comparing individual contribution. In the symmetric v5 model, contribution is measured through behavioral logs:

- Block placement ratio: did one participant place significantly more blocks than the other? (Dominant coder pattern)
- POV usage: did both participants enter Robot POV, or only one? (Self-organized division vs. joint experience)
- Log stamping ratio: did stamping divide evenly, or did one participant specialize in observation?
- Debug initiation: who initiated Debug Move 1/2/3 more often?

These patterns, analyzed alongside post-session individual knowledge tests, allow the study to examine whether natural role emergence in a symmetric design produces learning outcomes comparable to enforced role division — a genuine empirical contribution absent from v4.

Unity Assets

https://assetstore.unity.com/packages/3d/props/electronics/sci-fi-robot-helper-323875

